TTK4155
INDUSTRIAL AND EMBEDDED COMPUTER SYSTEMS DESIGN

Lab Lecture 4

23.10.2026

Tord Natlandsmyr

Lecture Overview

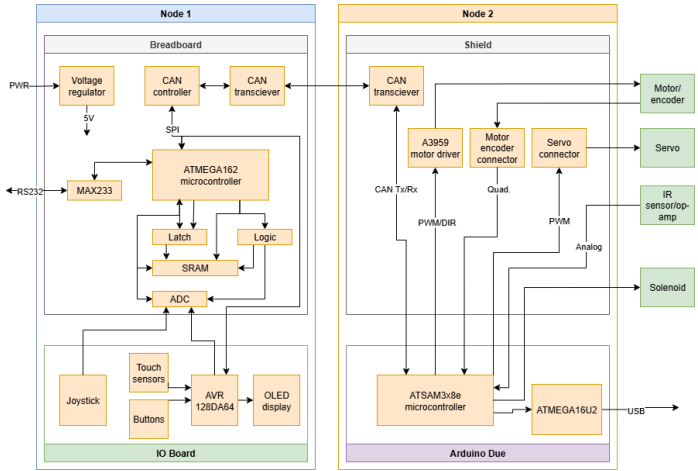
Recap 3
Arduino Due & PWM 6
Exercise 7:
Controlling servo and IR 9
Exercise 8:
Controlling motor and solenoid 20

Recap

Where are we?

Week	Exercise to be completed	Lab lecture
36	1: Initial assembly of microcontroller and RS-232	Lab Lecture 2, EL3
37	2: Address decoding and external RAM	
38	3: A/D conversion and joystick input	
39	4: OLED display and SPI part 1, IO buttons	
40	4: OLED display and SPI part 1, IO buttons	Lab Lecture 3, EL3
41	5: SPI part 2 and CAN controller	
42	6: Getting started with ARM cortex-M3 and communication between nodes	
43	Catch up week	**Lab Lecture 4, EL3**
44	7: Controlling servo and IR	
45	8: Controlling motor and solenoid	
46	Catch up week + Code delivery 13 Nov at 17:00	
47	Evaluation	

Interface with Game Board (green)



Arduino Due & PWM

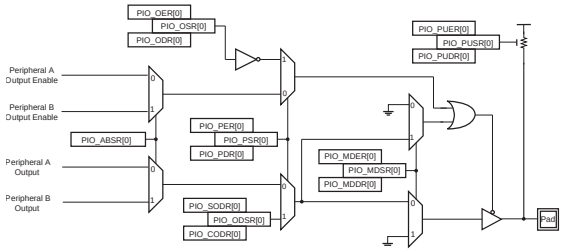
ATSAM3X8E Peripherals (Arduino Due)

- Multiplexed peripherals (ch 9)
- Controlled by PIO controller (ch 31)
  - Get familiar with fig 31-3
- Enable peripheral clock signal
  - `PMC->PMC_PCERn |= (1 << PIDn);`
- Disable PIO (ch 31.5.2)
  - `PIOc->PIO_PDR |= PIO_PDR_Pn;`
- AB Peripheral select (ch 31.5.3)
  - `PIOc->PIO_ABSR|= PIO_ABSR_Pn;`

Table 9-2. Multiplexing on PIO Controller A (PIOA)

I/O Line	Peripheral A	Peripheral B	Extra Function	Comments
PA0	CANTX0	PWML3		
PA1	CANRX0	PCK0	WKUP0[1]	
PA2	TIOA1	NANDRDY	AD0[2]	
PA3	TIOB1	PWMFI1	AD1/WKUP1[3]	

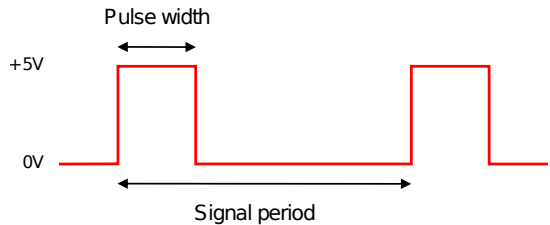
Figure 31-3. I/O Line Control Logic (Excerpt)



Also see `can.c` in the code handout

Pulse Width Modulation (PWM)

- A way for an MCU to generate an analog voltage from digital output
- Motor control: Enables multiple modes of control (analog, phase, polarity...)
- Signal easily flattened by capacitor as low-pass filter, or relying on the "slowness" of a system
- Average voltage $= 5V \cdot \text{Duty Cycle}$



Duty cycle = Pulse width/Signal period

Parameters:

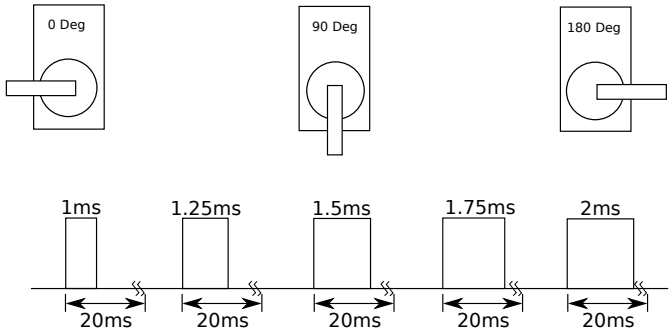
- Signal period/frequency
- Pulse width
- Duty cycle

Exercise 7:
Controlling servo and IR

Controlling the Servo

- Some servo systems can be controlled with PWM
- The angle of the servo motor is determined by the pulse width, 1.5 ms corresponding to the center position
- The servo is controlled using a 50 Hz PWM signal
- $\text{period} = T = \frac{1}{f} = \frac{1}{50\ \text{Hz}} = 20\ \text{ms}$
- By varying the pulse width we can control the angle of the motor
- The pulse width should not be outside the range of $0.9 - 2.1\ \text{ms}$
- Please inspect the signal with an oscilloscope to make sure you input the correct signal to avoid damage

180 deg Servo Positions and PWM



Information is encoded in the *pulse width*, not the duty cycle or PWM frequency. This is a quasi-standard that likely emerged from the hobby industry.

How to generate a PWM signal

Two alternatives:

- **Recommended**: PWM peripheral (datasheet ch 38)
  - Fewest lines of code and fewer registers needed.
  - Need to use the 5V level-shifted PIN44 and PIN45 on the shield.
- Timer Counter peripheral (datasheet ch 36)
  - Similar to how its done in node one for ADC clock: Counter with compare that resets pin.
  - Need to use the 5V level-shifted PWM2 pin on the shield
  - Some have experienced an unstable servo with this solution

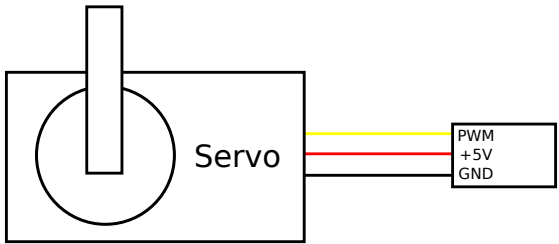
Standard Timer/Counter is not recommended, as Atmel has set up PWM using T/C in the PWM peripheral.

Generating PWM with the PWM peripheral

- Dedicated peripheral for generating PWM signals
- Important registers:
  - `PWM->PWM_CLK` / `REG_PWM_CLK`: PWM Clock register. Sets a prescaler, not required
  - `PWM->PWM_CH_NUM[n].PWM_CMR` / `REG_PWM_CMRn`: PWM Channel Mode Register
  - `PWM->PWM_CH_NUM[n].PWM_CPRD` / `REG_PWM_CPRDn`: PWM Channel Period Register
  - `PWM->PWM_CH_NUM[n].PWM_CDTY` / `REG_PWM_CDTYn`: PWM Channel Duty Cycle Register
  - `PWM->PWM_ENA` / `PWM_ENA: PWM`: Enable Register
- Multiple names for each register (/), defined in the `sam` config directory.

Servo Pinout

- After verifying spec compliance using oscilloscope, you may connect servo.
- Verify by measuring pulse width using the capture and cursor functions



IR Beam Score System

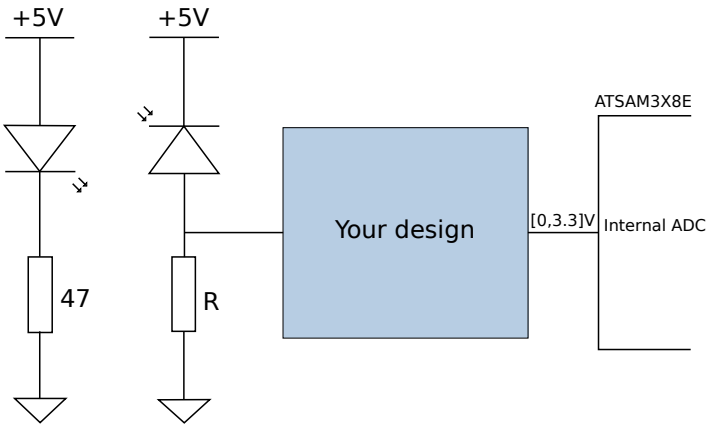


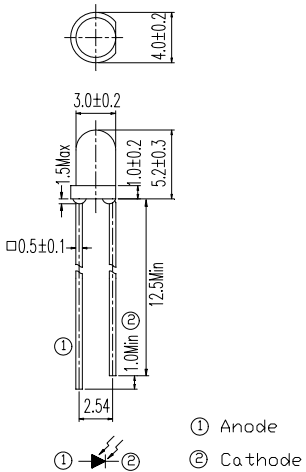
IR Photo Diode Receiver

IR Diode Transmitter

- The current through the photo-diode is dependent on the amount of light received
- To measure the current, we need a resistor to measure the voltage over
- Pick a resistor based on the photo-diode datasheet

IR Schematic





Connecting te IR Circuitry

- Use a voltage follower (op-amp).
- Use the small bread-board.
- Connect it as close to the photo-diode as possible to increase noise immunity.
- Check that the IR diode is working using a digital camera (e.g. mobile phone pointed directly towards the top of the diode).
- Do not connect to MCU before verifying voltage level compliance [0, 3.3]
  - You will use the internal ADC of the ATSAM3X8E (datasheet ch 43)
  - Find a pin with an ADC peripheral that is not used elsewhere.
  - Table 9-2 in the datasheet is a good starting point.

ADC Peripheral

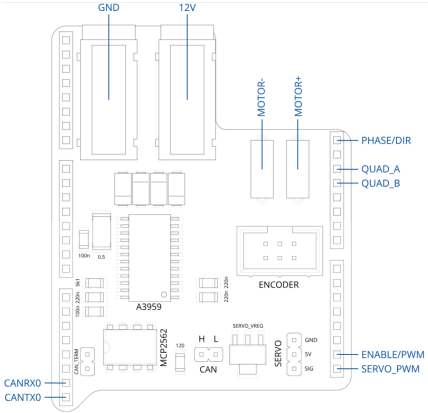
- Not A/B peripheral, but extra. See datasheet section 43.5.3 for activation.
- Multiple modes supported, single ended freerun mode is the easiest.
- Important registers:
  - `ADC_MR`: ADC Mode Register
  - `ADC_CHER`: ADC Channel Enable Register
  - `ADC_CR`: ADC Control Register
  - `ADC_CDR[n]`: ADC Channel n Data Register
  - `ADC_LCDR`: ADC Last Converted Data Register

Exercise 7 Tasks

- Use the node two to generate a PWM signal.
- Make sure your PWM signal is within the specifications of servo i.e. 0.9 to 2.1 ms.
- Use the joystick to control the pulse width of the PWM i.e. setpoint.
- Read the signal from an IR photo-diode. If required, use digital filter.
- Detect when the IR-beam is blocked and count the score
- **Ensure that the OpAmp circuit output does not exceed 3.3V before connecting it to Node 2**

Exercise 8:
Controlling motor and solenoid

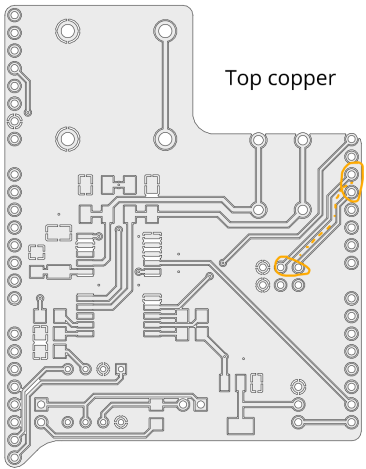
Controlling the Motor and Solenoid



- Connect large jacks to power supply, and small jacks to motor
- Implement an encoder signal reader on `ENABLE/PWM` and `PHASE/DIR`
- Create discrete controller on node 2 using motor interface drivers
- Create circuit for controlling solenoid from node 2
- Please read the shield schematic carefully!

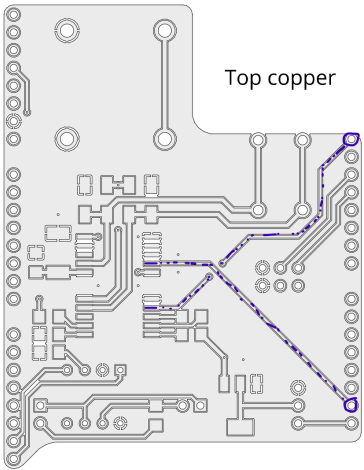
,

Reading the Motor Encoder



- Internal 32-bit counter, TC2 CH0
- Encoder mode: Channel is externally driven
- Important registers:
  - `TC_CV`: Current count value
  - `TC_BMR`: Channel mode register
  - `TC_CCR`: Channel control register
- Overflow vs physical end points
- Timer counters are everywhere: PWM, RTCs, interrupts, encoders... Learn them!

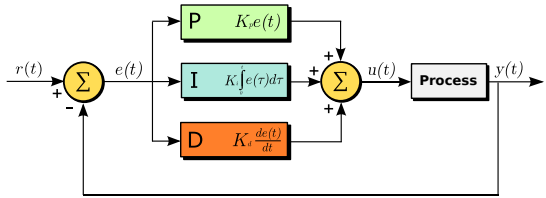
Allegro A3959 motor driver: Motor Actuation



- ENABLE: Activate motor
- DIR: ...with the given polarity
- $\rightarrow$ PWM on Enable for speed control, DIR for direction
- Duty cycle determines speed, motor acts as a low pass filter
- Should be able to reuse code from servo control

Position Controller

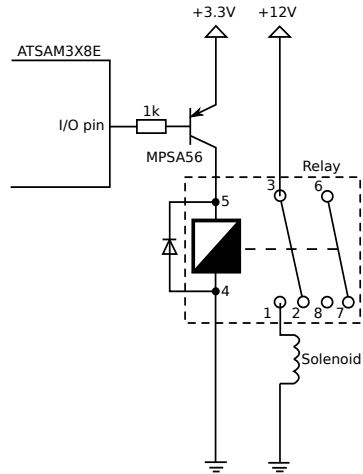
- The quadrature encoder will give an estimate of the motor position
- The error is the difference between indicated position and desired position (e.g. from joystick)
- Calculate the controller constants
- Sum all values and apply to motor output
- AVR221: Discrete PID Controller on tinyAVR and megaAVR Devices



- Regular timing is **important** for stability.
- Hint: Timer/Counter interrupt

Solenoid

- The solenoid requires about 12V to throw a solid punch
- Relays are a good option
- The ATSAM can not supply the current to drive the relay -> transistor
- Protective diode at the relay input
- If your node 2 is resetting, chances are you are not using the diode
- While 3.3V works, the relay works better at 5V. Use multiple channels on your lab supply, and **make sure you are not putting 5V on node 2 pins**



Exercise 8 Tasks

- Connect the ping-pong board and node 2
- Create a motor control driver and test with joystick input (sent from node 1).
- Create a position controller (minimum requirement) using feedback from the quadrature encoder.
- Connect the solenoid and use a button on the joystick to trigger.

Note from Anders

- Old Motor driver: error mode where it stops working and possibly gets hot.
  - If you have issues, as the TA's for an updated board
- ICE: unknown error mode where it stops working when both node 1 and 2 are connected. Don't do that.
- Servo 5V regulator: Gets quite hot, handle with care

Final Evaluation

- Week 47 within lab hours
- Short practical demonstration (~10min/group) of the entire system at your lab place.
- **Please implement cool additions to make it less tedious for us to evaluate**
- Visual inspection of your system
- Questions about system functionality and implementation choices
- Recommended: Record a short practical demonstration in case anything goes wrong
- Your code will be delivered prior to the demonstration
  - Review & plagiarism control
- Handing the lab kit back in good order is an requirement to get access to the final exam.
- More information will be posted closer to the evaluation

Questions?

- E-mail: tord.natlandsmyr@ntnu.no
- Slides are available on Canvas

Thank you for your attention!

...and good luck on the final evaluation!